NYAYMITRA 2.0 — Technical Architecture Report

Frontend, backend, database, AI/RAG, data flow, API, security, deployment, limitations

NYAYMITRA 2.0 — Technical Architecture Report

1. Overview

NYAYMITRA 2.0 is a self-contained, offline-first FastAPI web application with a vanilla-JavaScript frontend. It is designed to run on modest infrastructure (single process, SQLite) without external service dependencies by default.

2. Frontend architecture

Technology	Vanilla JavaScript, HTML5, CSS3 (no framework, no CDN, no Google Fonts)
Pages	Home, assistant (Q&A;), library (constitution search), departments, glossary, complaint wizard, saved lookup
Languages	English + Gujarati toggle (`` updated)
Accessibility	Keyboard navigation, visible focus ring, 44 px tap targets, `aria-live`, `aria-invalid`, `aria-busy`, media queries, `prefers-reduced-motion`, viewport meta
Security	Content Security Policy (no `unsafe-inline`/`unsafe-eval`), `nosniff`, `no-store` on personal-data responses, no cookies, no inline styles/scripts
Responsiveness	CSS breakpoints, no horizontal scrolling at 360 px
Static checks	`node --check web/app.js web/admin.js` pass

Frontend files: `web/index.html`, `web/admin.html`, `web/app.js`, `web/admin.js`, `web/styles.css`, `web/favicon.svg`, `web/manifest.webmanifest`.

3. Backend architecture

FastAPI application (factory `create_app`), Python 3.11.

``` backend/app/ main.py — app factory, startup, production preflight config.py — env-based NYAYMITRA_* settings, no hard-coded secrets security.py — role-based tokens, rate limiting, CSP/middleware middleware.py — security headers, request id, cache control deps.py — DB dependency, token auth errors.py — AppError, handlers logging_utils.py — PII redaction (redact, redact_and_truncate) api.py — OpenAPI mounting metrics.py — in-memory counters models.py — SQLAlchemy-ish row models ai/ — providers, RAG, guard, offline composer, registry routers/ — health, constitution, ai, complaints, documents, departments, feedback, admin services/ — constitution search, complaints, documents, departments, triage, glossary, seeding, ingestion, monitoring, retention, review, provider_health ```

4. Database structure

SQLite, file `data/nyaymitra.db` (gitignored). Four migrations (001–004), all reversible.
```

Key tables:

- `constitution_articles` — 9 articles (title, citizen explanation, source text `NULL` unless loaded, verification status, text status)
- `article_sources` — 15 legal sources
- `departments` — 11 departments
- `triage_guides` — 5 guides
- `glossary` — 18 terms
- `complaints` — reference, token hash, category, title, description, contact, consent, status
- `documents` — uploaded file metadata + stored path
- `query_logs` — redacted question text + SHA-256 hash
- `ai_answers` — provider, model, grounded/degraded, citations, token usage
- `audit_log`, `error_log`, `feedback`, `ingest_runs`
- `legacy_import_chunks` — 2,125 quarantined legacy rows (never served)

Integrity: `PRAGMA integrity_check = ok`; 4 migrations applied, 0 pending; 26 indexes present, 0 missing.

5. AI / RAG workflow

1. Triage match — deterministic keyword match over curated guides. 2. Retrieval — up to 4 constitution rows via FTS5 (BM25). 3. Prompt-injection scan — warns, does not block. 4. Provider call — live OpenAI-compatible model if configured, else deterministic offline composer. 5. Validation — reject uncited article references and unsafe legal advice; fallback to offline. 6. Persistence — redacted query log + answer record with token usage. 7. Response — answer, citations, verification status, warnings, disclaimers.

Safety: `validate_answer` rejects answers citing articles not retrieved, or containing guilt determinations, outcome guarantees, final decisions, impersonation, or false certainty. Gujarati/Devanagari digits normalised for article matching.

6. API structure

Base path `/api/v1`. Key endpoints:

- `GET /ready`, `GET /meta`
- `GET /constitution/articles`, `GET /constitution/articles/{id}`
- `POST /ai/explain` — question → answer (citations + disclaimers)
- `POST /complaints/preview`, `POST /complaints`, `GET /complaints/{ref}?token=`
- `POST /documents/upload`, `GET /documents/{id}/content?token=`
- `GET /departments`, `GET /glossary`
- `GET /admin/{stats,queries,answers,errors,audit,complaints,documents,...}` (token-gated)
- `POST /admin/backup`

7. Security design

- Role-based admin/monitor tokens (constant-time compare, SHA-256 stored)
- Rate limiting per endpoint class (read/write/AI/admin)
- Parameterized queries (SQL injection prevention)
- Output escaping, CSP, `nosniff`, `no-store` for personal-data responses
- PII redaction before local logging/storage

- Production preflight blocks unsafe startup (weak tokens, DB inside repo, proxy trust without origins)
- `pip-audit`: no known vulnerabilities

8. Deployment architecture

- `Dockerfile` — non-root user, health check on `/ready`
- `deploy/nyaymitra.service` — systemd unit with hardening
- `render.yaml` / `deploy/render.persistent.yaml` — Render deployment shape (free-tier demo; ephemeral disk noted)
- `.env.example` — environment template (no real secrets)
- Reverse proxy required for TLS termination (not implemented in-app)

9. Technical limitations

- Single-process SQLite; not for concurrent production load.
- No real browser, device, or screen-reader testing (jsdom only).
- No live AI model available in sandbox; offline fallback is default.
- Legal content not yet traced to official authoritative sources (0 of 9 verified).
- TLS termination must be handled by operator/reverse proxy.
- No self-service data deletion endpoint yet.

10. Repository

https://github.com/dhundhwana-arch/Nyaymitra-2.0 | branch `arena/812d6894-nyaymitra-2-0` | audited source snapshot `cc56d0e`

NYAYMITRA 2.0 — functional software prototype / Proof of Concept — subject to independent evaluation. Not legal advice; not a replacement for courts, lawyers, police, or government authorities. https://github.com/dhundhwana-arch/Nyaymitra-2.0 (branch arena/812d6894-nyaymitra-2-0, commit cc56d0e)